

Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez

Javier Alejandro Penagos Hernandez

Nicolas Castro Rivera

1 Executive Summary

Customer Churn is a problem about customer discontinue a company's services or products. Machine learning techniques can be applied with purpose of predict if a customer has probability of be churn, this information can help the company to take decisions for not lost clients. This work search explore different machine learning techniques to compare and get the best approach to solve this problem.

2 Dataset Download and Integrity Verification

2.1 Dataset Source and Acquisition

The Telco Customer Churn dataset was obtained from the public Kaggle page published by BlastChar and identified as originating from IBM Sample Data Sets. To reproduce the setup, download the complete `WA_Fn-UseC_-Telco-Customer-Churn.csv` file from <https://www.kaggle.com/datasets/blastchar/telco-customer-churn> and place it at `data/WA_Fn-UseC_-Telco-Customer-Churn.csv`. The Kaggle page does not specify a standard open-source license; this project uses the data for academic learning and analysis and acknowledges its source and original authors.

2.2 Dataset Characteristics

The CSV is 977,501 bytes (approximately 977.5 KB) and contains 7,043 customer records and 21 columns: 20 customer and service features, plus the binary `Churn` target. It has 1,869 `Yes` and 5,174 `No` records, indicating class imbalance.

2.3 Integrity Verification

The provider does not supply an official checksum. To calculate the SHA-256 hash of the local project copy, run `pythonsrc/w1_supervised_learning/dataset_info/checksum.py` from the repository root. The script reads the CSV under `data/` and prints its checksum:

```
88BE4B93FBE0CC83421AF1C503794C97C342ECA914C1576DB7C276E61D61358A
```

This value is a local reference because Kaggle does not provide an official checksum for comparison. To generate a structured dataset report, run `pythonsrc/w1_supervised_learning/dataset_info/dataset_info.py` from the repository root. It prints JSON with the file size, row and column counts, samples per **Churn** class, each column's data type and missing-value count, and numeric feature ranges. Both scripts use the Telco CSV by default; a different file can be supplied with `--csv`.

3 Exploratory Data Analysis (EDA)

This section explore the data, explain what represents each feature and respective type, the proportion in the values of numerical and categorical features, the relation between each feature with the target and visual representation of statistics obtained.

All process in this section can be explore in EDA notebook.

Feature	Description	Data type
customerID	Unique value that identify a customer	str
gender	Determines whether customer is Male or Female	str
SeniorCitizen	Determines whether a person is citizen or not	int64
Partner	Determines whether person has partner	str
Dependents	Determines whether customer has a dependent	str
tenure	Number of months has stayed with the company	int64
PhoneService	Whether customer has phone service	str
MultipleLines	Whether the customer has multiple lines, or one, or does not have phone service	str
InternetService	Internet service provider (Fiber optic, DSL or no)	str
OnlineSecurity	Whether customer has online security, or not, or does not have internet service	str
OnlineBackup	Whether customer has online backup, or not, or does not have internet service	str
DeviceProtection	Whether customer has device protection, or not, or does not have internet service	str
TechSupport	Whether customer has tech support, or not, or does not have internet service	str
StreamingTV	Whether customer has streaming tv, or not, or does not have internet service	str
StreamingMovies	Whether customer has streaming movies, or not, or does not have internet service	str
Contract	Contract term duration (Month-to-month, One year, Two year)	str
PaperlessBilling	Whether customer has paperless billing or not	str
PaymentMethod	Payment method (Electronic check, Mailed check, Bank transfer (automatic), Credit card)	str
MonthlyCharges	Value paid by customer monthly	float64
TotalCharges	Total value paid by customer	str
Churn	Whether the customer churned or not, stopped using the service last month.	str

Table 1: Dataset features description

3.1 Target Distribution

Of the 7,043 customers, 5,174 (73.46%) stayed and 1,869 (26.54%) churned. The majority-to-minority ratio is 2.77:1, so the target is moderately imbalanced. The class counts are summarized in Figure 1 below.

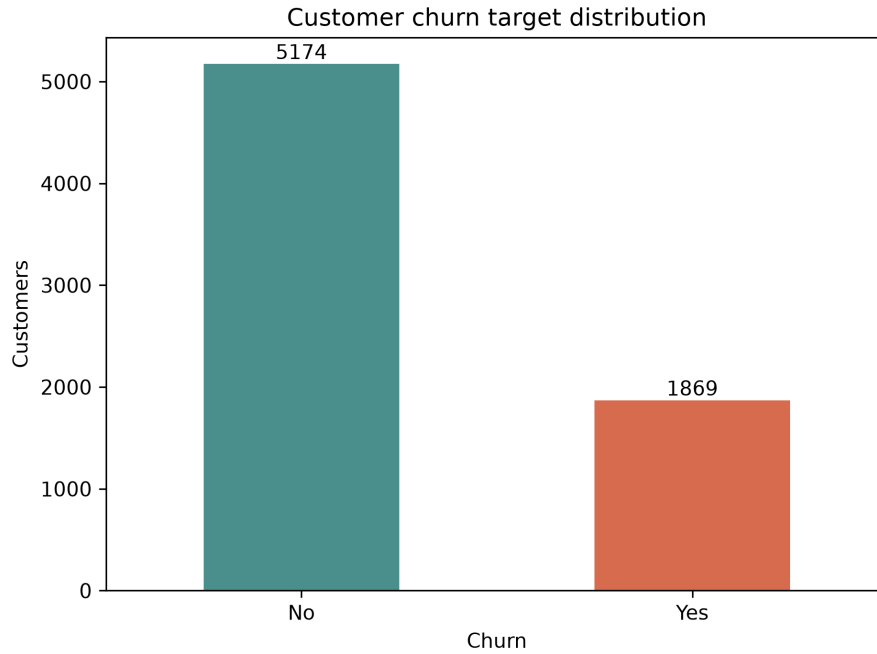


Figure 1: Distribution of customers by churn status.

3.2 Feature Statistics

Table 2 summarizes the non-binary numeric features: customer tenure and charges. The binary **SeniorCitizen** feature is excluded from these measurements.

Feature	N	Mean	SD	Min	Max
tenure (months)	7,043	32.37	24.56	0	72
MonthlyCharges	7,043	64.76	30.09	18.25	118.75
TotalCharges	7,032	2,283.30	2,266.77	18.80	8,684.80

Table 2: Summary statistics for numeric features.

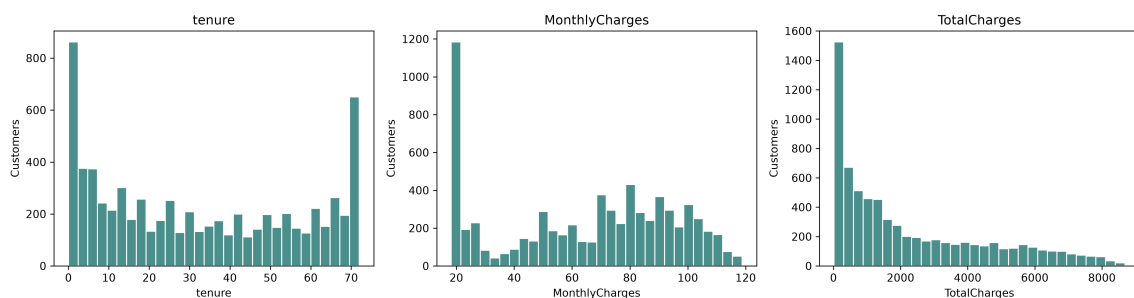


Figure 2: Histograms of tenure, monthly charges, and total charges.

3.3 Categorical Features

The categorical features describe customer demographics and household status, phone and internet services, service add-ons, streaming, contract and billing arrangements, and payment method. Each bar shows the percentage of all 7,043 customers in that category. The unique identifier `customerID` and target `Churn` are excluded here; the target distribution is shown separately in Figure 1.

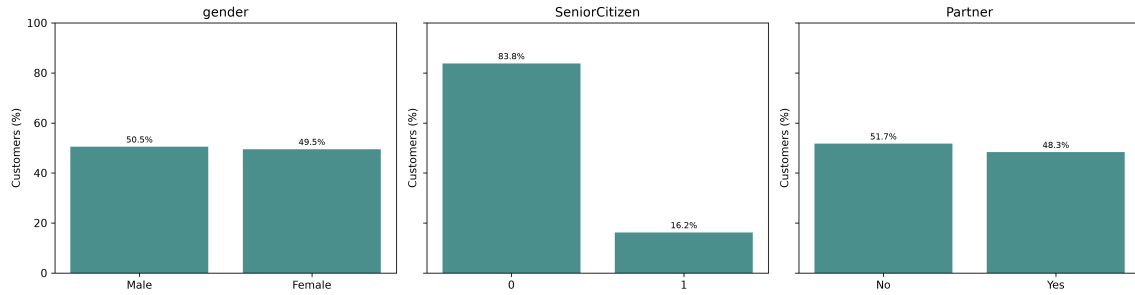


Figure 3: Distributions of customer profile features.

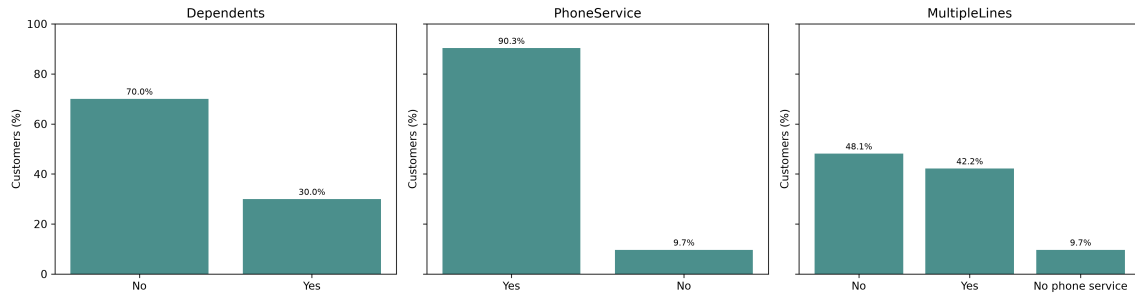


Figure 4: Distributions of household and phone service features.

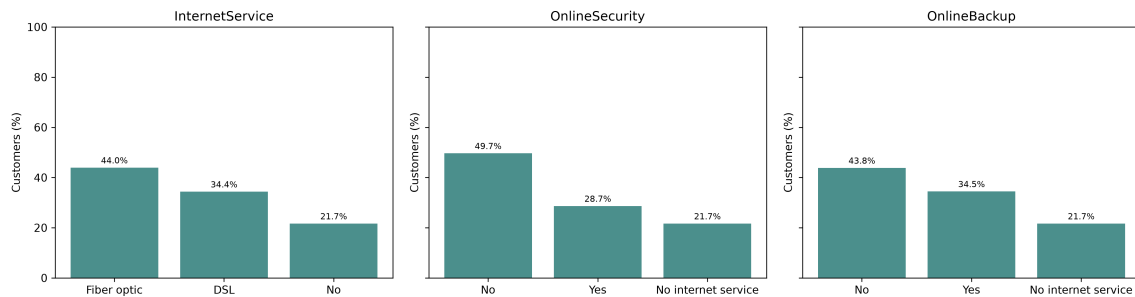


Figure 5: Distributions of internet service and online service features.

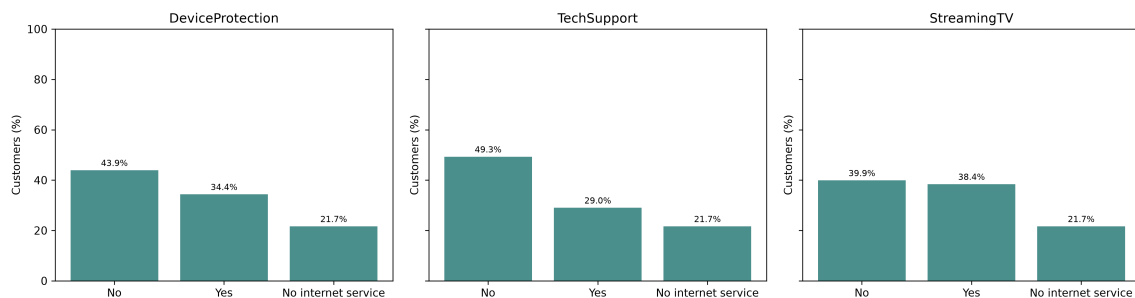


Figure 6: Distributions of device protection, technical support, and streaming TV.

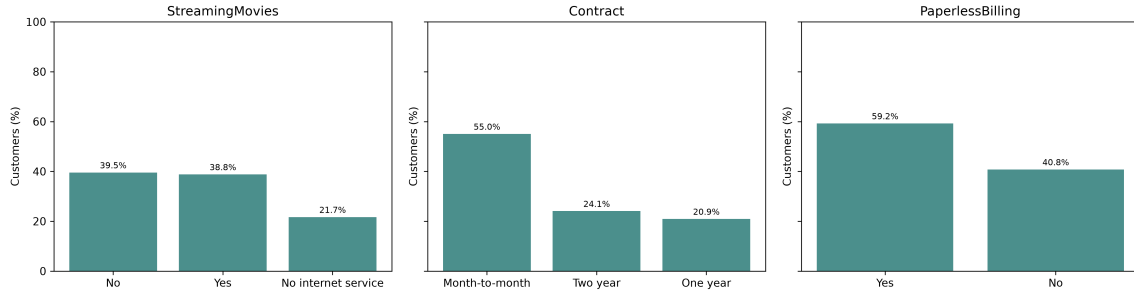


Figure 7: Distributions of streaming movies, contract, and paperless billing.

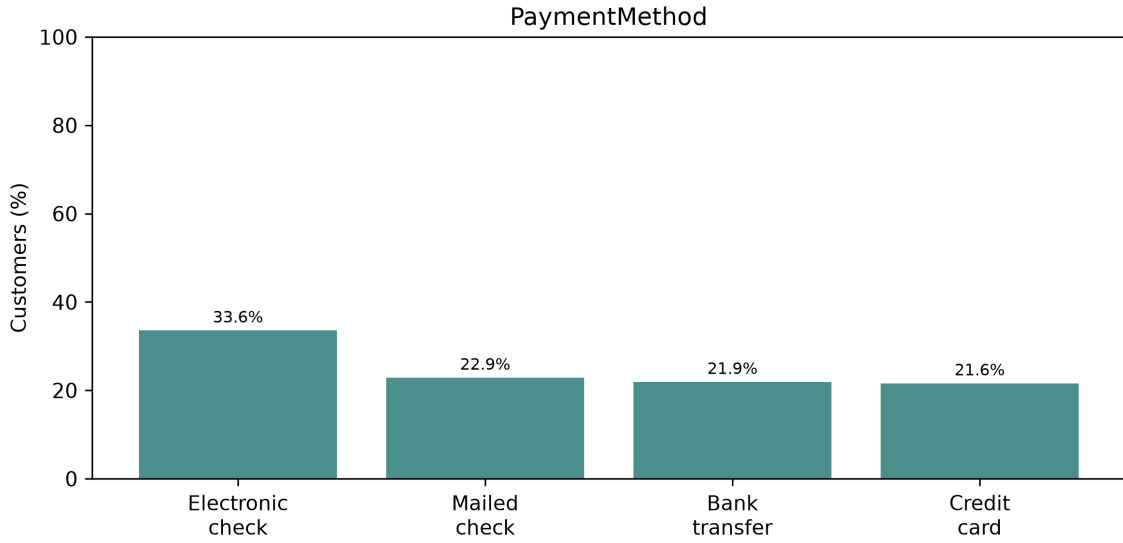


Figure 8: Distribution of customer payment methods.

3.4 Data Quality

The CSV contains no parsed missing cells or duplicate rows, and all 7,043 customer IDs are unique. However, 11 **TotalCharges** values are blank strings (0.16% of records). Is a little portion unusable. There are 7,032 values equal (99.84% of records)

3.5 Feature Relationships

These charts compare each feature with the churn target to identify differences between customers who stayed and those who left. The numerical-feature boxplots compare the distributions of tenure, monthly charges, and total charges across the two target classes. For categorical features, each pair of bars shows the percentage of customers within a category who stayed or churned; the two percentages sum to 100% for each category. These are descriptive associations and do not establish causation.

The numerical comparisons show that churned customers have a shorter median tenure (10 months versus 38) and higher mean monthly charges (74.44 versus 61.27). Among categorical features, churn is more common for month-to-month contracts (42.7%) than one-year (11.3%) or two-year contracts (2.8%), and for fiber-optic internet (41.9%) and electronic-check payments (45.3%).

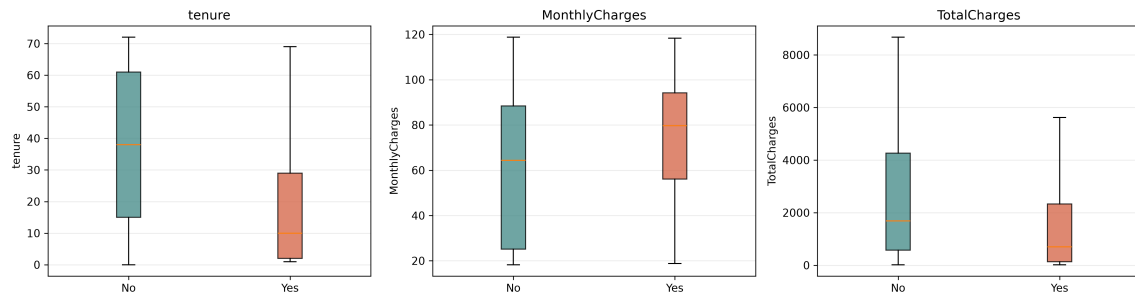


Figure 9: Numeric feature distributions by churn status.

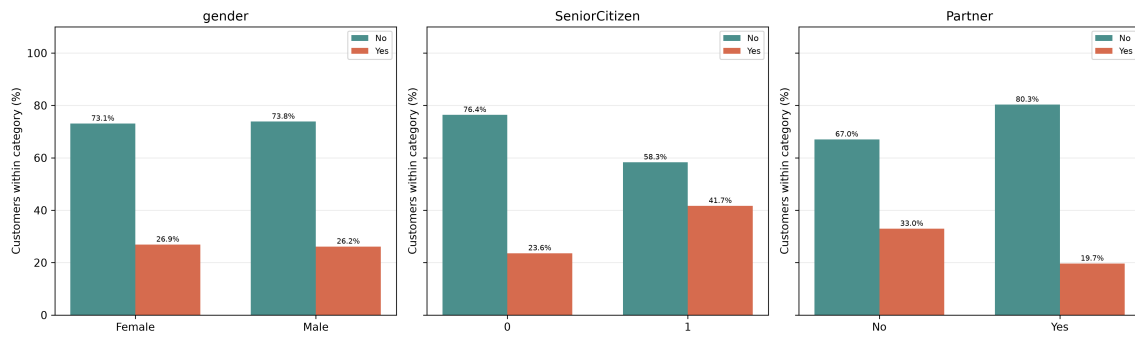


Figure 10: Churn rates by customer profile features.

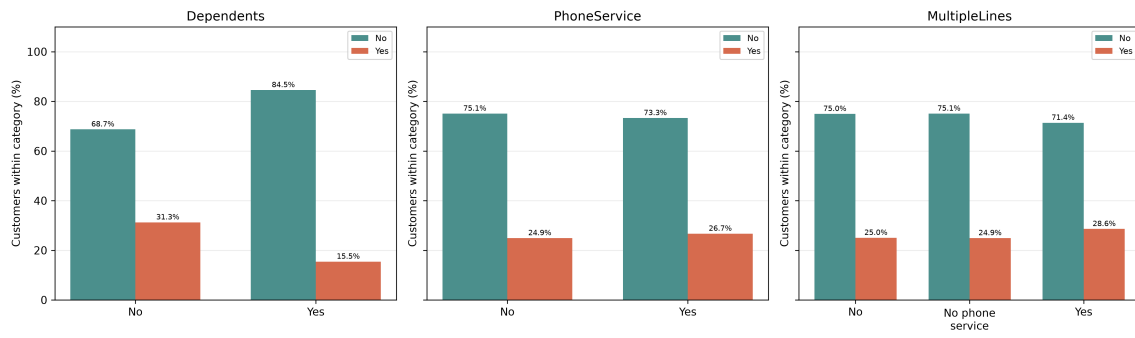


Figure 11: Churn rates by household and phone features.

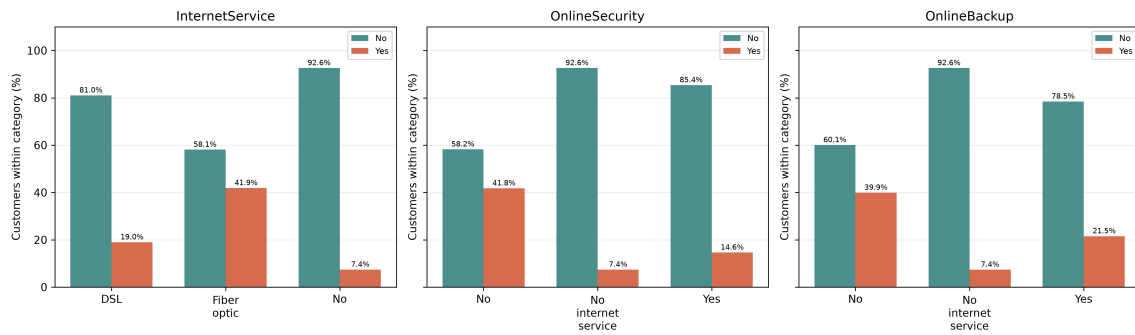


Figure 12: Churn rates by internet and online security features.

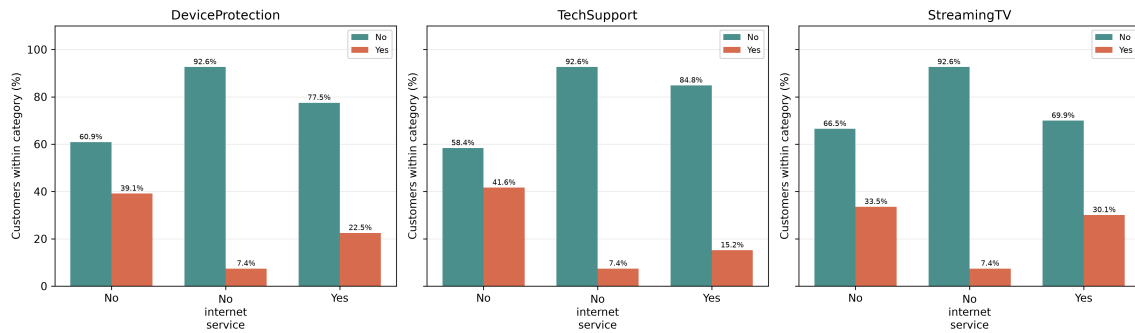


Figure 13: Churn rates by device protection, tech support, and streaming TV.

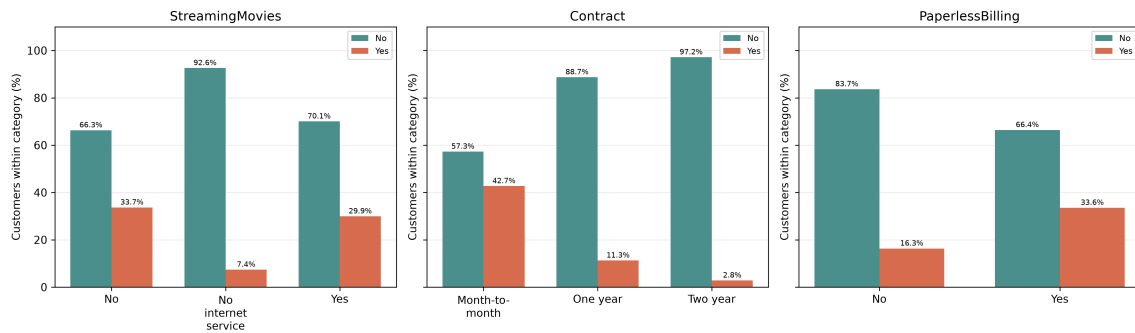


Figure 14: Churn rates by streaming movies, contract, and billing features.

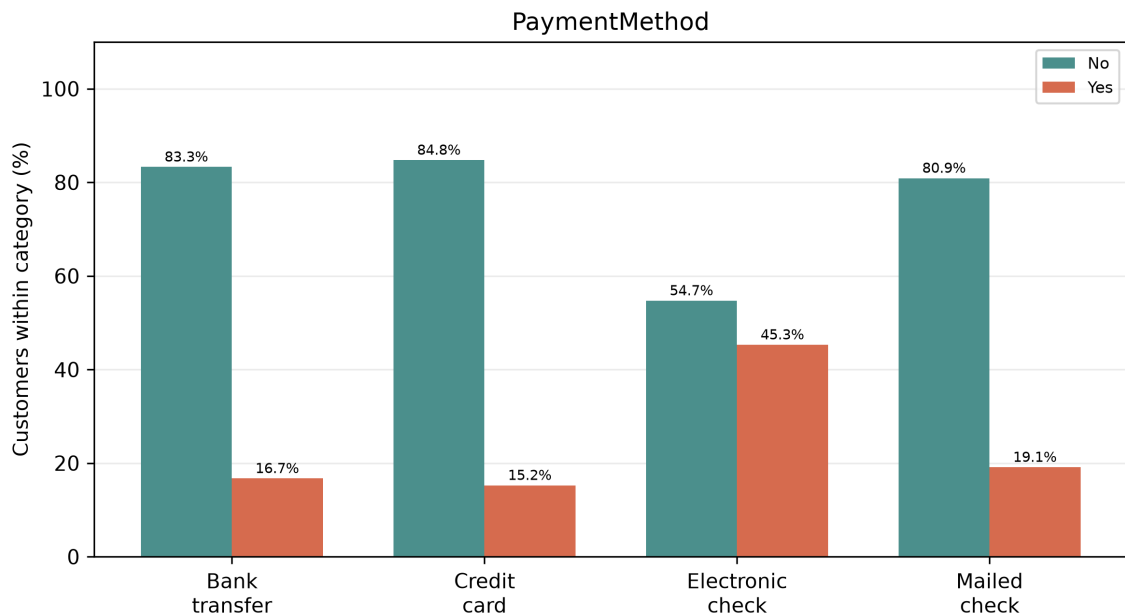


Figure 15: Churn rates by payment method.

4 Preprocessing Pipeline

The preprocessing pipeline is implemented by the preprocessing telco file. Consist in a Class with methods Methods that allow realise the preprocessing. This structure allow separate each step in the

processing of data and is possible add steps simply adding methods to the class. No data augmentation is applied because the nature of the dataset not has images, isn't necessary this technique.

4.1 Format Standardization

The pipeline selects the customer and service features used for modeling and excludes `customerID`, since it uniquely identifies each row and does not describe customer behavior. Categorical inputs remain in their original form until after splitting, when they are encoded using the training feature schema.

4.2 Missing Data Handling

`TotalCharges` is converted to numeric values using imputing. Its 11 blank entries in this dataset belong to customers with zero months of tenure, so they are imputed as zero. The target `Churn` is mapped to binary values, with `No`=0 and `Yes`=1. Any remaining rows with missing values are removed, but in this dataset there isn't that case because the 11 problematic samples where imputed. For the current dataset, all 7,043 rows remain after this step.

4.3 Feature Engineering

Two customer-level features are added. `NewCustomer` is set to 1 when tenure is zero and 0 otherwise. `AvgMonthlyCharges` is calculated as total charges divided by tenure for existing customers; it is set to zero when tenure is zero to avoid division by zero.

4.4 Train, Validation, and Test Split

The data is split before fitting the categorical encoder or scaler. A stratified split with random seed 42 assigns 70% of the records to training and divides the remaining 30% equally between validation and test, yielding approximately 70%/15%/15%. Stratification preserves the target class proportions in each subset. The validation arrays are available as `X_validation` and `y_validation` on the preprocessing object.

4.5 Encoding, Normalization, and Scaling

After splitting, categorical features are converted to one-hot indicators with `OneHotEncoder`. The encoder uses `handle_unknown=ignore` so unseen categories can be transformed using the training feature schema. The encoder and `StandardScaler` are fitted using training data only, then applied unchanged to validation and test data to prevent information leakage. Standard scaling is applied to the continuous features `tenure`, `MonthlyCharges`, `TotalCharges`, and `AvgMonthlyCharges`. The binary `SeniorCitizen` feature and one-hot indicator columns are not scaled.

4.6 Saved Splits

The transformed train, validation, and test arrays, their target labels, and the feature names are saved in compressed NumPy format at `data/telco_preprocessed_splits.npz`. The `preprocess()` method continues to return `X_train`, `X_test`, `y_train`, and `y_test` for compatibility with the existing model notebooks; the validation split is available as attributes and in the saved file.

5 Models

All four models below share the exact same experimental protocol, so that any difference in the final numbers can be attributed to the algorithm itself and not to inconsistencies in how it was trained or measured. Each model is developed in its own notebook under `src/w1_supervised_learning/models/`, consumes the identical train/validation/test arrays produced by `PreprocessingTelco`, and follows the same four stages: a hyperparameter search driven by Optuna’s tree-structured Parzen estimator sampler with median pruning [5], a decision-threshold search performed strictly on held-out predictions, a single refit on the full training split, and one evaluation on the test split. The test set is deliberately touched only once per model, at the very end, so that none of the reported numbers have been used, even indirectly, to make a modeling decision.

The search itself never optimizes accuracy. With 73.5% of customers staying, a model that always predicts “no churn” is already 73.5% accurate while being completely useless for retention purposes, so accuracy would reward the wrong behaviour during tuning. Instead, every search optimizes average precision, the area under the precision–recall curve, which is the metric of choice for imbalanced classification problems because it is sensitive to how well the model ranks the minority class rather than to how many labels happen to match [8]. Only after the best configuration is found is a decision threshold chosen, by scanning candidate cut-off points and keeping the one that maximizes F1 on validation-style predictions the model has not been directly fit to.

Table 3 lists the winning configuration found for each model. A pattern is already visible before looking at a single test metric: every search converged on a comparatively small, heavily regularized model rather than the largest one it was allowed to try. That is usually a sign that the extra capacity had nothing useful left to fit, a point the Results and Discussion section returns to.

Model	Selected configuration
Logistic Regression	<code>penalty=L2</code> , <code>C</code> ≈0.40, no class weighting
Random Forest	<code>n_estimators</code> =150, <code>max_depth</code> =10, <code>min_samples_leaf</code> =8, <code>max_features</code> =sqrt
XGBoost	<code>n_estimators</code> =200, <code>learning_rate</code> ≈0.043, <code>max_depth</code> =3, <code>reg_alpha</code> ≈7.4
Neural Network (MLP)	1 hidden layer of 48 units, dropout=0.35, batch normalization, 20 epochs

Table 3: Winning hyperparameter configuration per model, found with a 50-trial (30 for the MLP) Optuna search.

5.1 Logistic Regression

The linear model exists to answer a simple question: how much do the more elaborate models actually buy the project? Its search space covered L1, L2, and elastic-net penalties together with the inverse regularization strength `C`, and the winner was a fairly conservative L2 penalty with `C`≈0.40, meaning the search preferred to keep coefficients modest rather than let the model chase every fluctuation in the training data. Because every input was standardized beforehand, the resulting coefficients are directly comparable, and the largest ones line up with exactly the features that stood out earlier in the churn-relationship charts: contract length, tenure, and internet service type. The fact that a model with no interaction terms and no non-linearities lands so close to the ensembles below is itself an early hint about how much signal this dataset actually contains.

5.2 Random Forest

The forest searched over the number of trees, their maximum depth, the minimum samples required to split or to form a leaf, the feature-sampling strategy, and the split criterion [3]. The winning configuration used only 150 trees capped at depth 10, with a comparatively large minimum leaf size of 8. A Random Forest is not prone to overfitting simply by adding more trees, so if deeper, more numerous trees had helped, the search had every opportunity to pick them; instead it settled for a shallow, small forest, which again suggests the ceiling on this problem is set by the data rather than by the model's capacity.

5.3 XGBoost

Gradient boosting builds trees sequentially, each one correcting the residual errors of the ones before it, which in principle lets it capture more subtle patterns than bagging [4]. The search here covered ten hyperparameters at once, and the winning combination is telling: trees limited to depth 3, a cautious learning rate of about 0.043 spread over 200 boosting rounds, a strong L1 penalty (`reg.alpha`≈7.4) discouraging large leaf weights, and both row and column subsampling turned on. Every one of those choices dampens the model rather than empowering it. Interestingly, the search also settled on a `scale_pos_weight` of about 1.33, well below the dataset's true 2.77:1 imbalance ratio, meaning it found that correcting only partly for the imbalance, rather than fully rebalancing the loss, produced better-ranked probabilities.

5.4 Neural Network (MLP)

The multilayer perceptron is the only model built from scratch rather than imported from scikit-learn, using PyTorch [6] for both the architecture and the training loop. Optuna was allowed to search over one to four hidden layers of 16 to 128 units each, dropout, batch normalization, the learning rate, weight decay, the number of epochs, and the batch size. Out of that considerably larger design space, the winning network has a single hidden layer of 48 units (a 47→48→1 architecture), trained for only 20 epochs with dropout of 0.35. Given four layers and 128 units per layer were on the table, the network choosing the smallest allowed footprint is the clearest version of the pattern already seen in the two

tree-based models: more capacity was available, and it was not used, because there was nothing more to learn from these 47 encoded features.

6 Results and Discussion

6.1 Head-to-head comparison

Table 4 reports every model’s performance on its held-out test data, evaluated exactly once. Random Forest and XGBoost are effectively tied for the best average precision (0.672 and 0.671), Logistic Regression is a close third (0.658) despite being by far the simplest model, and the neural network trails the other three (0.631). No model reaches an accuracy of 80%, and the gap between the best and worst F1 score is under two points – the four models, built on genuinely different principles, all landed in nearly the same place. This lines up with the broader body of applied research on telecom churn prediction, where tree ensembles and linear baselines built on similar tabular features are consistently reported as strong, closely-matched performers [1].

Model	Avg. Precision	ROC-AUC	F1	Recall	Precision	Accuracy
Logistic Regression	0.658	0.845	0.631	0.758	0.541	0.764
Random Forest	0.672	0.841	0.623	0.673	0.580	0.783
XGBoost	0.671	0.844	0.630	0.705	0.569	0.780
Neural Network	0.631	0.843	0.625	0.757	0.532	0.759

Table 4: Test-set performance for all four models, each evaluated once on its held-out fold at its own tuned decision threshold.

The confusion matrices in Table 5 make the trade-off concrete. Logistic Regression and the neural network both catch around three out of every four churners (recall ≈ 0.76) but pay for it with more false alarms, while Random Forest is the most conservative model, missing more churners but rarely flagging a loyal customer by mistake. In a real retention campaign this is exactly the dial a business would want to turn: recall matters more when a retention offer is cheap, precision matters more when it is expensive, and the fact that four different algorithms land on four different points along that same trade-off, rather than one model dominating every column, says as much about the ceiling in the data as it does about any single model.

Model	Test size	True neg.	False pos.	False neg.	True pos.
Logistic Regression	1,057	595	181	68	213
Random Forest	1,057	639	137	92	189
XGBoost	1,057	626	150	83	198
Neural Network	1,409*	786	249	91	283

Table 5: Confusion matrix for each model on its own test fold. *See the testing note below regarding the neural network’s fold size.

6.2 On testing and reproducibility

No automated test suite (`pytest` or otherwise) exists yet for this workshop, which the team considers the most important gap to close before the next delivery. In its place, three weaker but real safeguards are already at work. First, all four notebooks import the same `PreprocessingTelco` class, so a bug in feature engineering or encoding would affect every model identically rather than silently favouring one of them. Second, the 5-fold stratified cross-validation used during every hyperparameter search is, in effect, twenty-five repeated end-to-end runs of the whole pipeline (five folds across fifty trials), and a broken step would have shown up as a crashed trial or a wildly inconsistent score rather than a clean result. Third, every random seed is fixed, so the numbers in this report are reproducible rather than a lucky draw.

That said, preparing this section surfaced a real inconsistency that a proper test would have caught automatically: the neural network notebook was executed before the preprocessing pipeline’s split ratios were finalized, so its cached test fold has 1,409 rows (20% of the data) instead of the 1,057 rows (15%) shared by the other three models. The metrics in Table 4 are each internally consistent, and the four models remain comparable in relative terms, but the neural network’s exact test numbers should be treated as provisional until that notebook is re-run against the current pipeline. This is precisely the kind of drift a small assertion on split sizes would have flagged the moment it happened, and it is now the first unit test on the team’s list.

6.3 What the correlations in the graphics are really showing

The EDA figures were built to compare each feature with the churn target one at a time, but looking across several of them side by side tells a more interesting story: many of these “independent” features are not independent at all, and that overlap is a large part of why every model above lands in the same narrow band of performance.

The clearest example sits in Figures 12 and 13. `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, and `StreamingMovies` each get their own chart, yet all six repeat almost exactly the same shape: customers with no internet service churn at about 7.4%, customers who have internet but skipped the add-on churn at roughly 40%, and customers who took the add-on churn somewhere in between. That is not six separate warning signs; it is the same underlying fact – whether a customer subscribes to internet service at all, and how invested they are in it – echoed six times through six differently named columns. A model does not gain six times the information from these variables; realistically it gains one strong signal wrapped in five correlated copies of itself.

A second, related pattern connects contract type, payment method, and tenure. Month-to-month customers churn at 42.7% against 2.8% for two-year contracts (Figure 14), and electronic check is both the only payment method above a 20% churn rate (45.3%, Figure 15) and, in practice, the payment method most associated with exactly the same short-contract, low-tenure customers. These three variables read as three independent risk factors, but they largely describe one recognizable group of newer, less-committed subscribers seen from three different angles, which is why adding a fourth or fifth correlated variable

rarely moves a model’s score by much.

The numeric relationships tell the same story from a different direction. The tenure and Monthly-Charges boxplots (Figure 9) show churners with a noticeably lower median tenure (10 months against 38) and higher median monthly charges (about 80 against 64), which is a genuine, usable signal – it is exactly what both the linear model’s coefficients and the tree-based models’ feature importances lean on most heavily. But the boxes overlap substantially: plenty of customers who stay have 15–29 months of tenure and \$56–88 in monthly charges, which is precisely the range where most churners also sit. There is no tenure or spending threshold that cleanly separates the two groups; there is only a region where churn becomes more likely, not certain.

Put together, this is the honest answer to why none of the four models crossed 80% accuracy or came close to it on any other metric. Three things compound against a higher ceiling: the target itself is imbalanced enough that a model gets 73.5% accuracy for free just by refusing to predict churn, so the 76–78% actually observed represents a real but modest lift, not a large one; the strongest predictors are correlated with each other rather than independent, so the model effectively has three or four usable signals dressed up as twenty columns instead of twenty genuinely different ones; and even those strongest signals overlap heavily between the two classes, so a meaningful share of customers are, on paper, indistinguishable regardless of which algorithm looks at them. That last point is confirmed by the hyperparameter search itself: Logistic Regression chose a moderate C , Random Forest stopped at a modest depth of 10, XGBoost preferred trees only three levels deep, and the neural network kept to a single 48-unit layer, even though every one of those searches was free to pick something larger. Four different algorithms, given the freedom to grow as complex as they wanted, all decided that more complexity was not worth it. That is a property of the dataset, not a shortcoming of any one model, and it sets a realistic, well-earned baseline for the deep sequential models the team will build in Workshop 2.

7 Conclusion

Workshop 1 set out to build a trustworthy supervised-learning baseline for churn prediction, and that goal was met. A preprocessing pipeline that splits before fitting anything, avoiding the leakage present in earlier drafts, now underpins a full exploratory analysis and four independently tuned models – Logistic Regression, Random Forest, XGBoost, and a small PyTorch neural network – compared under one identical protocol: the same data splits, the same search objective, and a single, honestly held-out evaluation. Random Forest and XGBoost lead on average precision, the linear baseline is a close and far more interpretable third, and the neural network, while competitive, did not benefit from the deeper architectures Optuna was free to try. That last result is itself useful: it says the team should not expect Workshop 2’s more sequential architectures to outperform this baseline by a wide margin unless they bring in information this feature set does not currently contain.

The work was successful not because any single model produced an outstanding number, but because the project can now explain, with evidence from the data itself, why none of them did: a moderately imbalanced target, a set of predictors that are more correlated with each other than the column count

suggests, and genuine overlap between the two classes in the strongest available features. That explanation, more than any single metric, is what makes this baseline usable going forward. The most immediate open items are to add an automated test suite that would have caught the neural network’s stale test-fold size on its own, to re-run that notebook against the finalized preprocessing pipeline, and to consider a business-driven decision threshold – built on the estimated cost of a missed churning versus a wasted retention offer, an approach with a track record in the churn literature [2] – rather than the F1-optimal one used here. Extending the feature set with account-level interactions, or revisiting the imbalance with a technique such as SMOTE [7] instead of scale-based weighting, are the most promising next steps for pushing past the ceiling this workshop uncovered.

References

- [1] Ahmad, A. K., Jafar, A., & Aljoumaa, K. (2019). Customer churn prediction in telecom using machine learning in big data platform. *Journal of Big Data*, 6, 28.
- [2] Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. *European Journal of Operational Research*, 218(1), 211–229.
- [3] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [4] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).
- [5] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2623–2631).
- [6] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- [7] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- [8] Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning* (pp. 233–240).